L15 — Sparse Matrices: Representation and Storage Techniques

Unit: 1 | **Chapter:** Pointers, Records & Advanced Arrays | **BT Level:** BT3 | **CO:** CO2, CO4

Learning Objectives

- Define a sparse matrix and identify when to use sparse representations
- Implement triplet (COO), CSR, and linked list representations
- Perform transpose of a sparse matrix efficiently
- Apply sparse matrix concepts to graph adjacency matrices

---

1. What is a Sparse Matrix?

A matrix is **sparse** if most of its elements are zero. There is no universal threshold, but a matrix with fewer than 1/3 non-zero elements is typically considered sparse.

Examples:

- Adjacency matrix of a sparse graph (roads, social networks)
- Finite element method matrices in physics simulations
- Natural Language Processing term-document matrices

Problem with dense storage: A 10,000 $\times$ 10,000 matrix requires $10^8 \times 4$ bytes $= 400$ MB for integers — even if only 1000 elements are non-zero!

---

2. Dense Matrix — Memory Waste

```
5×5 sparse matrix:
[0  0  3  0  0]
[0  0  0  0  0]
[7  0  0  0  2]
[0  0  0  0  0]
[0  1  0  0  0]

Non-zero elements: 3 (out of 25 = 12%)
Dense storage: 25 × 4 = 100 bytes
```

We can represent this with just the 3 non-zero values!

---

3. Triplet Representation (COO — Coordinate Format)

Store each non-zero element as a triplet `(row, col, value)`.

```
class SparseMatrix:
    def __init__(self, rows, cols):
        self.rows = rows
        self.cols = cols
        self.elements = []   # List of (row, col, value) tuples

    def insert(self, row, col, value):
        if value != 0:
            self.elements.append((row, col, value))

    def get(self, row, col):
        for r, c, v in self.elements:
            if r == row and c == col:
                return v
        return 0

    def display(self):
        print(f"Rows={self.rows}, Cols={self.cols}, Non-zeros={len(self
        for r, c, v in self.elements:
            print(f"  [{r}][{c}] = {v}")
```

Example:

```
sm = SparseMatrix(5, 5)
sm.insert(0, 2, 3)
sm.insert(2, 0, 7)
sm.insert(2, 4, 2)
sm.insert(4, 1, 1)
sm.display()
# Rows=5, Cols=5, Non-zeros=4
# [0][2] = 3
# [2][0] = 7
# [2][4] = 2
# [4][1] = 1
```

Memory: 3 × (number of non-zeros) values stored vs m × n for dense.

Operation	Time Complexity
Insert	O(1)
Lookup	O(nnz) where nnz = non-zeros
Iterate all	O(nnz)

---

4. Transposing a Sparse Matrix

To transpose: swap (row, col) → (col, row), then sort by new row.

```
def transpose(sm):
    result = SparseMatrix(sm.cols, sm.rows)
    for r, c, v in sm.elements:
        result.elements.append((c, r, v))   # Swap row and col
    # Sort by row, then col for canonical form
```

```
    result.elements.sort(key=lambda x: (x[0], x[1]))
    return result
```

---

5. Compressed Sparse Row (CSR) Format

CSR is the most widely used sparse format for arithmetic operations (matrix-vector multiply).

Three arrays:

- `values[]` — non-zero values in row-major order
- `col_idx[]` — column index of each non-zero value
- `row_ptr[]` — index in `values[]` where each row starts (length = rows + 1)

Example:

```
Matrix:
[0  0  3  0  0]
[0  0  0  0  0]
[7  0  0  0  2]
[0  0  0  0  0]
[0  1  0  0  0]

values  = [3,    7, 2,    1   ]
col_idx = [2,    0, 4,    1   ]
row_ptr = [0, 1, 1, 3, 3, 4]
          ↑   ↑  ↑  ↑  ↑  ↑
         r0  r1  r2  r3  r4  end
```

`row_ptr[i]` to `row_ptr[i+1]-1` gives the range in `values/col_idx` for row i.

```
class CSRMatrix:
    def __init__(self, rows, cols, values, col_idx, row_ptr):
        self.rows = rows
        self.cols = cols
        self.values = values
        self.col_idx = col_idx
        self.row_ptr = row_ptr

    def get(self, i, j):
        for k in range(self.row_ptr[i], self.row_ptr[i + 1]):
            if self.col_idx[k] == j:
                return self.values[k]
        return 0

    def matrix_vector_multiply(self, x):
        """Compute A * x"""
        result = [0.0] * self.rows
        for i in range(self.rows):
            for k in range(self.row_ptr[i], self.row_ptr[i + 1]):
                result[i] += self.values[k] * x[self.col_idx[k]]
        return result
```

Operation	CSR Time
Row access	O(nnz per row)
Matrix-vector multiply	O(nnz)
Column access	O(nnz)

---

6. Linked List Representation

Each row is a linked list of non-zero nodes:

```
Node: (col, value, next_ptr)

Row 0: → (2, 3, NULL)
Row 1: → NULL
Row 2: → (0, 7, →) → (4, 2, NULL)
Row 3: → NULL
Row 4: → (1, 1, NULL)
```

```
class SparseNode:
    def __init__(self, col, val):
        self.col = col
        self.val = val
        self.next = None

class LinkedSparseMatrix:
    def __init__(self, rows, cols):
        self.rows = rows
        self.cols = cols
        self.row_heads = [None] * rows

    def insert(self, row, col, val):
        if val == 0:
            return
        new_node = SparseNode(col, val)
        # Insert in order of column
        if self.row_heads[row] is None or self.row_heads[row].col > col
            new_node.next = self.row_heads[row]
            self.row_heads[row] = new_node
        else:
            curr = self.row_heads[row]
            while curr.next and curr.next.col < col:
                curr = curr.next
            new_node.next = curr.next
            curr.next = new_node
```

---

7. Format Comparison

Format	Storage	Insert	Lookup	Row Traverse	Matrix-Vec Multiply
Dense	$O(m \times n)$	O(1)	O(1)	O(n)	$O(m \times n)$

Triplet (COO)	O(nnz)	O(1)	O(nnz)	O(nnz)	O(nnz) with sorting
CSR	O(nnz + m)	O(nnz)	O(nnz/m)	O(nnz/m)	**O(nnz)**
Linked List	O(nnz)	O(nnz/m)	O(nnz/m)	O(nnz/m)	O(nnz)

---

8. Applications

Domain	Example
Graph Algorithms	Adjacency matrix of sparse graph (few edges)
Linear Systems	Finite element, finite difference methods
Machine Learning	TF-IDF matrices, word embeddings
PageRank	Web link graph (billions of pages, sparse)
Recommendation Systems	User-item rating matrices

Summary

- A **sparse matrix** has mostly zero entries — dense storage wastes memory.
- **Triplet (COO):** Simple, O(nnz) storage; good for construction and format conversion.
- **CSR:** Most efficient for row-oriented operations and matrix-vector multiplication.
- **Linked list:** Good for dynamic insertion/deletion of non-zeros.
- Sparse representations are foundational in scientific computing, graphs, and ML.

References

- Cormen et al., *Introduction to Algorithms*, Appendix (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.2 (Springer, 2020)